5–7 LPA Placement Preparation

Questions 16–30 — C++ | Brute Force + Optimized

Font sizes: 18 pt title • 16 pt question headings • 14 pt body/code • 1.5 line spacing

16. Reverse Words in a String

Example: Input: "I love coding" → Output: "coding love I"

Brute Force — Extra vector:

```
string s = "I love coding";
vector<string> words;
string word = "";
for(int i = 0; i <= s.length(); i++) {
if(i == s.length() || s[i] == ' ') {
if(word != "") {
words.push_back(word);
word = "";
}
} else {
word = word + s[i];
}
}
for(int i = words.size() - 1; i >= 0; i--)
cout << words[i] << " ";
```

Complexity: Time O(n), Space O(n).

Optimized — End se scan:

```
string s = "I love coding";
int end = s.length() - 1;
while(end >= 0) {
while(end >= 0 && s[end] == ' ') end--;
if(end < 0) break;
int start = end;
```

```
while(start >= 0 && s[start] != ' ') start--;
for(int i = start + 1; i <= end; i++)
cout << s[i];
cout << " ";
end = start - 1;
}
```

Complexity: Time O(n), Extra Space O(1).

17. Count Vowels and Consonants

Example: "Hello" → Vowels = 2, Consonants = 3

Approach:

```
string s = "Hello";
int vowels = 0, consonants = 0;
for(int i = 0; i < s.length(); i++) {
char ch = s[i];
if(ch == 'a' || ch == 'e' || ch == 'i' ||
```

ch == 'o' || ch == 'u' ||

ch == 'A' || ch == 'E' || ch == 'I' ||

ch == 'O' || ch == 'U')

```
vowels++;
else if((ch >= 'a' && ch <= 'z') ||
```

(ch >= 'A' && ch <= 'Z'))

```
consonants++;
}
cout << "Vowels = " << vowels << endl;
cout << "Consonants = " << consonants;
```

Complexity: Time O(n), Space O(1).

18. Count Frequency of Characters

Example: "hello" → h=1, e=1, l=2, o=1

Brute Force:

```
for(int i = 0; i < s.length(); i++) {
bool already = false;
for(int j = 0; j < i; j++) {
if(s[i] == s[j]) {
already = true;
break;
}
}
if(already) continue;
int count = 0;
for(int j = 0; j < s.length(); j++)
if(s[i] == s[j]) count++;
cout << s[i] << " -> " << count << endl;
}
```

Complexity: O(n²).

Optimized — Frequency Array:

```
int freq[256] = {0};
for(char ch : s)
freq[(unsigned char)ch]++;
for(int i = 0; i < 256; i++)
if(freq[i] > 0)
cout << char(i) << " -> " << freq[i] << endl;
```

Complexity: O(n) time, O(1) fixed ASCII space.

19. Find Duplicate Characters in a String

Example: "programming" → repeated characters include r, g, m

Brute Force:

```
for(int i = 0; i < s.length(); i++) {
bool duplicate = false;
for(int j = 0; j < i; j++) {
```

```
if(s[i] == s[j]) {
duplicate = true;
break;
}
}
if(duplicate) continue;
int count = 0;
for(int j = 0; j < s.length(); j++)
if(s[i] == s[j]) count++;
if(count > 1)
cout << s[i] << " ";
}
```

Complexity: O(n²).

Optimized:

```
int freq[256] = {0};
for(char ch : s)
freq[(unsigned char)ch]++;
for(int i = 0; i < 256; i++)
if(freq[i] > 1)
cout << char(i) << " ";
```

Complexity: O(n) time, O(1) fixed ASCII space.

20. Remove Duplicate Characters from a String

Example: "programming" → "progamin"

Brute Force:

```
string result = "";
for(int i = 0; i < s.length(); i++) {
bool found = false;
for(int j = 0; j < result.length(); j++) {
if(s[i] == result[j]) {
found = true;
```

```
break;
}
}
if(!found)
result = result + s[i];
}
cout << result;
```

Complexity: O(n²).

Optimized:

```
bool visited[256] = {false};
string result = "";
for(char ch : s) {
if(!visited[(unsigned char)ch]) {
result = result + ch;
visited[(unsigned char)ch] = true;
}
}
cout << result;
```

Complexity: O(n) time, O(1) fixed ASCII space.

21. Check if Two Strings are Anagrams

Example: "listen" and "silent" → Anagram

Brute Force — Manual sorting:

```
if(a.length() != b.length()) {
cout << "Not Anagram";
return 0;
}
for(int i = 0; i < a.length(); i++) {
for(int j = i + 1; j < a.length(); j++) {
if(a[i] > a[j]) {
char temp = a[i];
```

```
a[i] = a[j];
a[j] = temp;
}
if(b[i] > b[j]) {
char temp = b[i];
b[i] = b[j];
b[j] = temp;
}
}
}
if(a == b) cout << "Anagram";
else cout << "Not Anagram";
```

Complexity: O(n²).

Optimized — Frequency:

```
int freq[256] = {0};
for(char ch : a)
freq[(unsigned char)ch]++;
for(char ch : b)
freq[(unsigned char)ch]--;
bool anagram = true;
for(int i = 0; i < 256; i++) {
if(freq[i] != 0) {
anagram = false;
break;
}
}
if(anagram) cout << "Anagram";
else cout << "Not Anagram";
```

Complexity: O(n) time, O(1) fixed ASCII space.

22. Find the First Non-Repeating Character

Example: "swiss" → w

Brute Force:

```
for(int i = 0; i < s.length(); i++) {
int count = 0;
for(int j = 0; j < s.length(); j++)
if(s[i] == s[j]) count++;
if(count == 1) {
cout << s[i];
break;
}
}
```

Complexity: O(n²).

Optimized:

```
int freq[256] = {0};
for(char ch : s)
freq[(unsigned char)ch]++;
for(char ch : s) {
if(freq[(unsigned char)ch] == 1) {
cout << ch;
return 0;
}
}
cout << "No non-repeating character";
```

Complexity: O(n) time, O(1) fixed ASCII space.

23. Find the Largest Element in an Array

Example: 10 50 20 80 30 → Largest = 80

Brute Force:

```
for(int i = 0; i < n; i++) {
bool largest = true;
for(int j = 0; j < n; j++) {
if(arr[j] > arr[i]) {
largest = false;
```

```
break;
}
}
if(largest) {
cout << arr[i];
break;
}
}
```

Complexity: O(n²).

Optimized — Single Scan:

```
int largest = arr[0];
for(int i = 1; i < n; i++) {
if(arr[i] > largest)
largest = arr[i];
}
cout << largest;
```

Complexity: O(n) time, O(1) space.

24. Find the Smallest Element in an Array

Brute Force:

```
for(int i = 0; i < n; i++) {
bool smallest = true;
for(int j = 0; j < n; j++) {
if(arr[j] < arr[i]) {
smallest = false;
break;
}
}
if(smallest) {
cout << arr[i];
break;
}
```

```
}
```

Complexity: O(n²).

Optimized:

```
int smallest = arr[0];
for(int i = 1; i < n; i++) {
if(arr[i] < smallest)
smallest = arr[i];
}
cout << smallest;
```

Complexity: O(n) time, O(1) space.

25. Find the Second Largest Element in an Array

Example: 10 50 20 80 30 → Second Largest = 50

Brute Force:

```
int largest = arr[0];
for(int i = 1; i < n; i++)
if(arr[i] > largest)
largest = arr[i];
int second = -2147483648;
for(int i = 0; i < n; i++)
if(arr[i] != largest && arr[i] > second)
second = arr[i];
cout << second;
```

Complexity: O(n), two passes.

Optimized — Single Pass:

```
int largest = INT_MIN;
int second = INT_MIN;
for(int i = 0; i < n; i++) {
if(arr[i] > largest) {
second = largest;
```

```
largest = arr[i];
}
else if(arr[i] > second && arr[i] != largest) {
second = arr[i];
}
}
if(second == INT_MIN)
cout << "Second largest does not exist";
else
cout << second;
```

Complexity: O(n) time, O(1) space.

26. Find the Second Smallest Element in an Array

Brute Force:

```
int smallest = arr[0];
for(int i = 1; i < n; i++)
if(arr[i] < smallest)
smallest = arr[i];
int second = INT_MAX;
for(int i = 0; i < n; i++)
if(arr[i] != smallest && arr[i] < second)
second = arr[i];
cout << second;
```

Complexity: O(n), two passes.

Optimized — Single Pass:

```
int smallest = INT_MAX;
int second = INT_MAX;
for(int i = 0; i < n; i++) {
if(arr[i] < smallest) {
second = smallest;
smallest = arr[i];
}
```

```
else if(arr[i] < second && arr[i] != smallest) {
second = arr[i];
}
}
if(second == INT_MAX)
cout << "Second smallest does not exist";
else
cout << second;
```

Complexity: O(n) time, O(1) space.

27. Find the Sum of Array Elements

Example: 10 20 30 40 → Sum = 100

Brute Force / unnecessary nested work:

```
int sum = 0;
for(int i = 0; i < n; i++) {
for(int j = i; j <= i; j++)
sum = sum + arr[j];
}
```

Complexity: O(n^2).

Optimized:

```
int sum = 0;
for(int i = 0; i < n; i++)
sum = sum + arr[i];
cout << sum;
```

Complexity: O(n) time, O(1) space.

28. Reverse an Array

Example: 1 2 3 4 5 → 5 4 3 2 1

Brute Force — Extra Array:

```
int rev[5];
for(int i = 0; i < n; i++)
rev[i] = arr[n - 1 - i];
for(int i = 0; i < n; i++)
cout << rev[i] << " ";
```

Complexity: O(n) time, O(n) space.

Optimized — Two Pointer:

```
int left = 0;
int right = n - 1;
while(left < right) {
int temp = arr[left];
arr[left] = arr[right];
arr[right] = temp;
left++;
right--;
}
for(int i = 0; i < n; i++)
cout << arr[i] << " ";
```

Complexity: O(n) time, O(1) space.

Built-in reverse() use nahi kiya.

29. Check if an Array is Sorted

```
Example: 1 2 3 4 5 → Sorted; 1 3 2 4 5 → Not Sorted
```

Brute Force:

```
bool sorted = true;
for(int i = 0; i < n; i++) {
for(int j = i + 1; j < n; j++) {
if(arr[i] > arr[j]) {
sorted = false;
break;
}
}
```

```
if(!sorted) break;
}
if(sorted) cout << "Sorted";
else cout << "Not Sorted";
```

Complexity: O(n²).

Optimized — Adjacent Comparison:

```
bool sorted = true;
for(int i = 0; i < n - 1; i++) {
if(arr[i] > arr[i + 1]) {
sorted = false;
break;
}
}
if(sorted) cout << "Sorted";
else cout << "Not Sorted";
```

Complexity: O(n) time, O(1) space.

30. Remove Duplicates from an Array

Example: 1 2 2 3 3 4 → 1 2 3 4

Brute Force:

```
int result[6];
int k = 0;
for(int i = 0; i < n; i++) {
bool found = false;
for(int j = 0; j < k; j++) {
if(result[j] == arr[i]) {
found = true;
break;
}
}
if(!found) {
```

```
result[k] = arr[i];
k++;
}
}
for(int i = 0; i < k; i++)
cout << result[i] << " ";
```

Complexity: O(n²) time, O(n) space.

Optimized — Visited Array:

```
bool seen[101] = {false};
for(int i = 0; i < n; i++) {
if(!seen[arr[i]]) {
cout << arr[i] << " ";
seen[arr[i]] = true;
}
}
```

Complexity: O(n) time, O(range) space.

```
For general values, unordered_set gives average O(n) time
and O(n) space.
```

■ Interview Revision — Most Important Patterns

Frequency Array → Q18, Q19, Q20, Q21, Q22

Single Pass → Q23, Q24, Q25, Q26, Q27

Two Pointer → Q16, Q28

Adjacent Comparison → Q29

Hashing / Visited → Q18–22, Q30

□ Interview Revision — Most Important Patterns

Frequency Array → Q18, Q19, Q20, Q21, Q22

Single Pass → Q23, Q24, Q25, Q26, Q27

Two Pointer → Q16, Q28

Adjacent Comparison → Q29

Hashing / Visited → Q18–22, Q30